Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Döngü kullanımında sık yapılan hatalar (quiz değerlendirmesinden notlar)
- Yığın (stack) veri yapısı: son giren ilk çıkar (LIFO, last in first out)
- Yığının kullanım yerleri: tarayıcıda geri gitme, metin düzenleyicide geri alma (undo), fonksiyon çağrıları, özyineleme (recursion), aritmetik ifadeler
- Dizi ile yığın gerçekleştirme: `STACK` yapısı, `initialize`, `empty`, `full`, `push`, `pop`, `peek`
- Ara (infix) ifadeyi son (postfix) ifadeye çevirme
- Son ifadeyi yığınla hesaplama
- Derleyicinin iki yığınla (işlenen ve işleç yığınları) aritmetik ifade hesaplaması
- Parantez denetimi: sayı ve sıra kontrolü
- İki yığınla geri alma ve yineleme
- Kuyruk (queue) veri yapısı: ilk giren ilk çıkar (FIFO, first in first out)
- Dairesel kuyruk (circular queue) ve bağlı listeyle kuyruk

Hocanın Özellikle Vurguladığı Kısımlar

- Adım sayısı belli olmayan aramalarda `for` yerine `while` kullanın. Aranan bulununca döngüden çıkın. Bir milyon elemanlı dizide ilk adımda bulduysanız bir milyon kez dönmenin anlamı yoktur.
- Dizi üzerinde dönen bir `while` döngüsünde önce dizinin sonuna gelinip gelinmediğini kontrol edin (`i < n`), sonra aranan koşulu (`a[i] != x`). Aranan eleman hiç olmayabilir.
- Not değil öğrenmek önemlidir. Ödev ve quizleri kendi başınıza yapın.
- Yığında `push` etmeden önce dolu olup olmadığını (`full`), `pop` etmeden önce boş olup olmadığını (`empty`) kontrol edin.
- `initialize`, `display` gibi işleri ayrı fonksiyonlara koyun. Kodu okuyan biri işin nerede yapıldığını hemen görür.
- Yapıyı (struct) fonksiyona değer olarak değil adres olarak gönderin. Böylece bütün içerik boşu boşuna kopyalanmaz.
- `return` olan bir `if`'ten sonra `else` yazmak gerekmese de yazın. Kodu ileride değiştiren biri `return`'ü kaldırırsa program yanlış çalışmaz.
- Yığın için sürekli `realloc` ve `free` yapmayın. Yer bir kez ayrılır ve hep o bölge kullanılır.
- Stack overflow hatası, bitmeyen özyinelemede fonksiyon çağrıları için kullanılan yığının dolmasından kaynaklanır.
- İnfix'ten postfix'e çevirmede kapa parantez gelince aç paranteze kadar bütün işleçler çıkarılır. Aç parantez çıkarılır ama postfix ifadeye yazılmaz.
- Yalnızca aç ve kapa parantezlerin sayısının eşit olması yetmez. Sıraları da doğru olmalıdır.
- Konuları kendi kendinize deneyerek pekiştirin. Dairesel kuyruk fonksiyonlarını (`enqueue`, `dequeue`, `isFull`, `isEmpty`) internete bakmadan kendiniz yazın.

Kısa Tekrar Notları

- Yığın: son giren ilk çıkar (LIFO). Kuyruk: ilk giren ilk çıkar (FIFO).
- Yığın soyut bir veri tipidir. Gerçekte bir dizi kullanılır ama diziye yalnızca yığın kurallarıyla erişilir.
- `top`: bir sonraki elemanın yerleşeceği gözü gösterir.
  - Başlangıçta `top = 0`.

  - Boş: `top == 0`.
  - Dolu: `top == MAXSIZE`.
- `push`: `items[top] = x; top++;`
- `pop`: `top--; x = items[top];`
- `peek`: en üstteki elemanı döndürür ama `top`'u değiştirmez.
- `pop` edilen eleman dizide silinmez. `top` aşağı indiği için bir sonraki `push` o elemanın üzerine yazar.
- İşleç öncelikleri:
  - Yığına koyarken (push): `(` en yüksek, sonra `* /`, en düşük `+ -`.
  - Yığından çıkarırken (pop): `* /` en yüksek, sonra `+ -`, en düşük `(`.
- `3 + 5 * 4 / 2 - 6` ifadesinin postfix karşılığı: `3 5 4 * 2 / + 6 -`.
- `3 + 5 * (4 + 2 * 7) / 6` ifadesinin postfix karşılığı: `3 5 4 2 7 * + * 6 / +`. Değeri 18'dir.
- Postfix hesaplama: sayıyı yığına koy. İşleç gelince en üstteki iki sayıyı çek, işlemi yap, sonucu yığına geri koy.
- Parantez denetimi: `(` gelince push, `)` gelince pop. Boş yığından pop edilmeye çalışılırsa ya da sonda yığın boş değilse ifade hatalıdır.
- Dairesel kuyrukta yalnızca `front` ve `rear` hareket ettirilir. Elemanlar kaydırılmaz.

Detaylı Açıklamalar

Quiz Değerlendirmesinden Döngü Hataları

- **Hata 1:** Adım sayısı belli olmayan bir aramada `for` kullanıp aranan bulununca döngüden çıkmamak. Örneğin `a[i] == x` olunca `bul = 1` yapılıyor ama döngü dizinin sonuna kadar devam ediyor. Doğrusu `while` kullanıp aranan bulununca çıkmaktır.
- **Hata 2:** `while (a[i] != x)` gibi yalnızca aranan koşula bakmak. Aranan eleman hiç yoksa döngü dizinin dışına taşar. Önce `i < n`, sonra `a[i] != x` kontrol edilmelidir.
- İkinci sınıf ikinci dönemde artık adım sayısı belli olan ve olmayan döngülerde ne yapılacağı biliniyor olmalıdır.

Yığın (Stack) Nedir?

- Günlük bilgisayar kullanımında farkına varmadan çok kullanılan bir veri yapısıdır.
- Örnekler:
  - **Web'de gezinme:** gazete → yazarlar → bir yazar sayfasına gidildi. Geri tuşuna basınca önce yazarlar sayfasına, sonra gazeteye dönülür. Doğrudan gazeteye atlanmaz.
  - **Metin düzenleyicide geri alma (undo):** "Ali" yazıldıktan sonra geri alma, son yapılan işlemden başlayarak sırayla siler.
  - **Fonksiyon çağrıları:** `main`, `F1`'i; `F1`, `F2`'yi çağırır. `F2` bitince `F2`'yi çağıran yere (`F1`), `F1` bitince `main`'e dönülür. Doğrudan `main`'e atlanmaz.
  - **Özyineleme:** fonksiyonlar yığın kullanarak bulunduğu noktadan adım adım geri döner.
  - **Aritmetik ifadeler:** `3 + 4 * 5` ifadesinde derleyici soldan okur ama önce çarpmayı yapmalıdır. Toplamayı unutmamak için yığın kullanılır.
- Veri yapıları belli algoritmaların ihtiyaçlarına göre tasarlanır:
  - Dizide istenen göze doğrudan erişilir.
  - Bağlı listede her eleman kendinden sonrakini gösterir. Böylece bellekte dağınık duran bilgiler birbirini kaybetmez.
  - Yığında her adımda yalnızca en son bulunulan noktayla iş yapılır ve hep bir önceki duruma dönülür. Aradan bir elemana erişilmez.
- Yığın doğrusal (lineer) bir veri yapısıdır.
- Donanımda da bir yığın bölgesi vardır. Bizim kullandığımız ise soyut (abstract) bir veri tipidir: normal bir dizi, yalnızca yığın kurallarıyla kullanılır.

Dizi ile Yığın Gerçekleştirme

- İki temel işlem vardır:
  - `push`: gelen elemanı yığının üstüne koyar.
  - `pop`: yığının en üstündeki elemanı alır.

- Örnek: `top` 0'ı gösterir. `push(A)` ile A 0. göze yerleşir, `top` 1 olur. `push(B)` ile B 1. göze yerleşir, `top` 2 olur. `pop` ile `top` 1'e iner ve B alınır. Sonra `push(C)` gelirse C, 1. göze yerleşir.
- Yığının en az iki bilgisi olmalıdır: işlemlerin yapılacağı dizi ve yığında nerede olunduğunu gösteren `top`.
- Dizi dinamik olarak da ayrılabilir. Her durumda yığının bir en büyük eleman sayısı vardır.

Yapı ve yardımcı fonksiyonlar (ekrandan, 00:40:30):

```
#define MAXSIZE 100
typedef struct
{
    int items[MAXSIZE];
    int top;
} STACK;

void initialize(STACK *s)
{
    s->top = 0;
}

int empty(STACK *s)
{
    if (s->top == 0)
        return 1;
    else
        return 0;
}

int full(STACK *s)
{
    if (s->top == MAXSIZE)
        return 1;
    else
        return 0;
}
```

- Son eleman `MAXSIZE - 1`'inci göze yerleşir. Bu durumda `top` `MAXSIZE`'ı gösterir. Bu göz yığına ait değildir. Bu yüzden `top == MAXSIZE` ise yığın doludur.
- Bir öğrenci dolu koşulu için `top == MAXSIZE - 1` önerdi. Hoca düzeltti: `top` `MAXSIZE - 1` iken o göze hâlâ eleman yerleşebilir.
- `empty` ve `full` yığını değiştirmediği halde parametre adres olarak gönderilir. Değer olarak gönderilseydi yapının bütün içeriği fonksiyona kopyalanırdı. Adres gönderilince yalnızca referans gider. Bu daha verimlidir.

`push` ve `pop` (ekrandan, 01:05:10):

```
int push(int x, STACK *s)
{
    if (full(s))
        return 0;           /* yığın dolu, push yapılamadı */
    else
    {
        s->items[s->top] = x;
        s->top++;
        return 1;           /* push başarılı */
    }
}

int pop(STACK *s, int *x)
{
    if (empty(s))
        return 0;           /* yığın boş, pop yapılamadı */
```

```
    else
    {
        --s->top;
        *x = s->items[s->top];
        return 1;           /* pop başarılı */
    }
}
```

- `push` ve `pop` işlemin başarılı olup olmadığını döndürür. Çağıran yer buna göre işlem yapabilir.
- `pop`, başarı durumu yerine çıkan elemanı da döndürebilir (`return x`). Hangisinin seçileceği tasarıma bağlıdır. Burada çıkan eleman `x` göstericisi ile dışarı verilir.
- `pop`'ta `top` en üstteki elemanın bir üstünü gösterdiği için önce `top` azaltılır, sonra eleman alınır.
- `push`'ta atama sırasında `s->items[s->top++] = x;` da yazılabilir. Hoca dikkat çekmek için ayrı satırda yazdı.

`peek` İşlemi

- `pop` ile eleman yığından alınır. Aslında eleman dizide silinmez. `top` aşağı indiği için bir sonraki `push` o elemanın üzerine yazar.
- `peek`, yığının en üstündeki elemanı `top`'u değiştirmeden döndürür. Yığında bir değişiklik yapmaz.
- Örneğin yığında A, B, C varsa `peek` C'yi döndürür ama C yığında kalır.
- Fonksiyon çağrısı bitince dönüş adresi artık gerekmez. Bu durumda `pop` edilir. En üstteki elemanı yalnızca görmek gerekiyorsa `peek` kullanılır.
- `peek` bir değer döndürür, doğru/yanlış değil. Hoca `peek` fonksiyonunu yazmadı, öğrencilere bıraktı.

Dinamik Bellek Sorusu

- Soru: Pop edilen eleman için yer `free` edilse ve diziden çıkarılsa daha iyi olmaz mı?
- Cevap: Hayır. Yığın bölgesinin büyüklüğü ihtiyaca göre bir kez belirlenir ve hep o bölge kullanılır.
- Sürekli `free` ve `realloc` yapmak verimli değildir. `realloc` arkada boş yer yoksa bütün veriyi başka yere taşır. Bu zaman kaybıdır.
- Kabul edilebilir kullanım: iş bittiğinde fazla kalan yer için bir kez `realloc` yapmak. Örneğin quizdeki renk sorusunda en fazla k/3 renk için yer ayrıldıysa ve az renk kullanıldıysa iş sonunda bir kez küçültülebilir.

`if-return` Sonrasında `else` Yazmak

- `full(s)` doğruysa `return 0` ile fonksiyondan çıkılır. `else` yazılmasa da program doğru çalışır.
- Ama projede kodu sonra değiştiren biri `return`'ü kaldırıp yerine başka bir şey yazarsa, `else` olmadığı için akış aşağıya devam eder. Program yanlış çalışır.
- Bu sık rastlanan bir sorundur. Kodun anlaşılırlığı ve doğruluğunu garanti altına almak için `else` açıkça yazılmalıdır.

Ara (Infix) İfadeyi Son (Postfix) İfadeye Çevirme

- İnfix ifadede işleçler sayıların arasındadır: `3 + 5 * 4 / 2 - 6`.
- Postfix ifadede işleçler işlem sırasına göre sayıların arkasına yazılır: `3 5 4 * 2 / + 6 -`.
- Bu ifadede işlem sırası: önce 5×4, sonra $/2$, sonra $3+$, en son -6.
- Çevirme için bir işleç yığını (operator stack) kullanılır.

İşleç öncelikleri (ekrandan, 01:31:00):

Öncelik	Yığına koyarken (push)	Yığından çıkarırken (pop)
En yüksek	`(`	`* /`

Öncelik	Yığına koyarken (push)	Yığından çıkarırken (pop)
Orta	* /	+ -
En düşük	+ -	(

- Aç parantez, yığından çıkarılırken en düşük önceliklidir. Kapa parantez gelmeden parantezin içindeki işlemler bitmemiştir.
- Kural:
  - Sayı gelince doğrudan çıkışa yazılır.
  - İşleç gelince yığının üstüne `peek` ile bakılır. Üstteki işlecin önceliği gelen işlecinkinden büyük veya eşitse üstteki işleç çıkarılıp çıkışa yazılır. Bu, artık çıkarılamayan bir işleç kalana kadar sürer. Sonra gelen işleç yığına konur.
  - `)` gelince `(`'a kadar bütün işleçler çıkarılıp çıkışa yazılır. `(` da çıkarılır ama yazılmaz.
  - İfade bitince yığında kalan bütün işleçler sırayla çıkarılır.

Örnek 1: `3 + 5 * 4 / 2 - 6`

Gelen	İşlem	Yığın (alt → üst)	Çıkış
3	yaz		3
+	push	+	3
5	yaz	+	3 5
*	+ çıkamaz, push	+ *	3 5
4	yaz	+ *	3 5 4
/	* eşit öncelikli, çıkar; + çıkamaz; push	+ /	3 5 4 *
2	yaz	+ /	3 5 4 * 2
-	/ çıkar; + eşit öncelikli, çıkar; push	-	3 5 4 * 2 / +
6	yaz	-	3 5 4 * 2 / + 6
son	kalanları çıkar		3 5 4 * 2 / + 6 -

Örnek 2: `3 + 5 * (4 + 2 * 7) / 6`

Gelen	İşlem	Yığın (alt → üst)	Çıkış
3	yaz		3
+	push	+	3
5	yaz	+	3 5
*	push	+ *	3 5
(	push	+ * (	3 5
4	yaz	+ * (	3 5 4
+	(çıkamaz, push	+ * (+	3 5 4
2	yaz	+ * (+	3 5 4 2
*	+ çıkamaz, push	+ * (+ *	3 5 4 2
7	yaz	+ * (+ *	3 5 4 2 7
)	(’a kadar çıkar	+ *	3 5 4 2 7 * +
/	* eşit öncelikli, çıkar; + çıkamaz; push	+ /	3 5 4 2 7 * + *
6	yaz	+ /	3 5 4 2 7 * + * 6
son	kalanları çıkar		3 5 4 2 7 * + * 6 / +

- Postfix ifade işlem sırasını gösterir: önce 2×7, sonra $4+$, sonra $5\times$, sonra $/6$, en son $3+$.
- Derste bir öğrenci sordu: çarpıdan sonra 4 gelince, bölü işaretini beklemeden çarpı çıkarılamaz mıydı? Hoca cevapladı: bu örnekte doğru, ama algoritma standart olsun diye her işleç bir sonraki işleç gelene kadar yığında bekletilir. Aksi hâlde üs alma gibi yeni işleçler eklenince algoritma özel durumlarla dolar. Ayrıca çarpı çıkınca toplamanın çıkıp çıkamayacağını bilmek için yine sonraki işleci beklemek gerekir.

Postfix İfadeyi Hesaplama

- Bu sefer sayılar (değerler) için bir yığın kullanılır.

- Kural:
  - Sayı gelince yığına konur.
  - İşleç gelince yığının en üstündeki iki değer çekilir, işlem yapılır, sonuç yığına geri konur.
  - Sıraya dikkat: önce çekilen sağ işlenen, sonra çekilen sol işlenendir. Örneğin bölmede alttaki değer üsttekine bölünür ($90/6$).
  - İfade bitince yığında kalan değer sonuçtur. Örneğin `x = ...` ise bu değer bellekte x'in yerine yazılır.

`3 5 4 2 7 * + * 6 / +` için (ekrandan, 01:41:40):

Gelen	İşlem	Yığın (alt → üst)
3, 5, 4, 2, 7	push	3 5 4 2 7
*	$2 \times 7 = 14$	3 5 4 14
+	$4 + 14 = 18$	3 5 18
*	$5 \times 18 = 90$	3 90
6	push	3 90 6
/	$90/6 = 15$	3 15
+	$3 + 15 = 18$	18

- Gerçek ifadelerde sayılar yerine değişkenler olur (`A + B * C / D - E` gibi). Derleyici değişkenlerin değerlerini bellekten bulup aynı işi yapar.
- Atama işareti (=) bu işlemlere katılmaz. Derleyici sağ taraftaki işlemin sonucunu sol taraftaki değişkenin bellekteki yerine yazar.

Derleyicinin İki Yığınla Hesaplaması

- Derleyici aynı anda iki yığın kullanır:
  - İşlenen yığını (operand stack): değerler için
  - İşleç yığını (operator stack): işaretler için
- Bir işleç yığından çıkınca işlenen yığınının en üstündeki iki değer çekilir, işlem yapılır, sonuç işlenen yığınına geri konur.

Örnek: `5 + 4 * 2 - 1`

Gelen	İşlenen yığını	İşleç yığını	Açıklama
5	5		
+	5	+	
4	5 4	+	
*	5 4	+ *	* daha öncelikli, + çıkamaz
2	5 4 2	+ *	
-	13	-	* çıkar: $4 \times 2 = 8$. + çıkar: $5 + 8 = 13$
1	13 1	-	
son	12		$13 - 1 = 12$

Örnek: `3 * (5 + 4 / 1) - 2`

- 3, *, (, 5, +, 4 sırayla yerleşir. / gelince + çıkamaz, / yığına konur. 1 yerleşir.
-) gelince / çıkar: $4/1 = 4$. + çıkar: $5 + 4 = 9$. (çıkar, işlem yapılmaz.
- - gelince * çıkar: $3 \times 9 = 27$. - yığına konur. 2 yerleşir.
- Sonda - çıkar: $27 - 2 = 25$.

Örnek: `3 * (5 + 4 * 2 - 1)`

- Parantez içinde - gelince * çıkar ($4 \times 2 = 8$), + de eşit öncelikli olduğu için çıkar ($5 + 8 = 13$). (çıkamaz, - yığına konur.
-) gelince - çıkar: $13 - 1 = 12$.
- Sonda * çıkar: $3 \times 12 = 36$.

- Bu örnek, parantez içindeki işlemlerin de kendi önceliklerine göre çıktığını gösterir.
- Derleyici her işlecin önceliğini bir tabloda (örneğin bir struct'ta) tutar. Örneğin + ve - için 1, * ve / için 2. Gelen işlecin önceliği yığındakinden büyükse yığındaki çıkamaz.
- C'de kaydırma (shift), ++, -- gibi daha pek çok işleç vardır. Hepsinin önceliği tutulur.

Parantez Denetimi

- Derleyiciler ve Excel gibi programlar parantez hatalarını yığınla bulur.
- Yalnızca aç ve kapa parantez sayısını saymak yetmez. Örneğin)(gibi bir dizilişte sayılar eşittir ama sıra yanlıştır.
- Kural: (gelince push,) gelince pop.
- Hata durumları:
  -) gelince yığın boşsa (pop edilemiyorsa) parantez sırası yanlıştır ya da fazla kapa parantez vardır.
  - İfade bitince yığın boş değilse fazla aç parantez vardır. Yığın boşalana kadar pop edilerek kaç tane fazla olduğu bulunabilir.
- İfade bittiğinde yığın boşsa ve hiçbir pop hata vermediyse parantezler eşit sayıda ve doğru sıradadır.

Geri Alma ve Yineleme

- Soru: Geri alınan bir işlem tekrar istenirse (ileri, redo) ne yapılır?
- Cevap: İki yığın kullanılır. Geri alınan işlem ilk yığından çıkarılır ve ikinci yığına konur. İleri denince ikinci yığından en son geri alınan işlem verilir. Pek çok metin düzenleyici bu iş için iki yığın kullanır.

Kuyruk (Queue)

- Kuyruk, yığının "kardeşi" sayılabilir. Farklı bir mantıktır ama yine bilgilere belli bir düzene göre erişilir. Aradan bir elemana erişilmez.
- İlk giren ilk çıkar (FIFO). Hizmet kuyruğun başındakine (front) verilir. Yeni gelen kuyruğun sonuna (rear) yerleşir.
- Kullanım yerleri:
  - Ağ üzerindeki ortak yazıcı: çıktılar geliş sırasına göre basılır.
  - Giriş/çıkış tamponları (I/O buffer)
  - İşlemci zamanlaması (CPU scheduling), disk zamanlaması (disk scheduling)
  - İşletim sistemleri ve ağlarda kaynak paylaşımı
  - Banka sıra sistemi: yeni gelen numara kuyruğa eklenir, ekrandaki numaraya hizmet verilir.
- İki temel işlem vardır:
  - `enqueue`: kuyruğun sonuna eleman ekler.
  - `dequeue`: kuyruğun başındaki elemanı çıkarır.
- `isFull` ve `isEmpty` ile geçersiz işlemler engellenmelidir. Örneğin boş kuyruktan eleman çıkarmaya veya dolu kuyruğa eleman eklemeye çalışmak.
- `peek`: kuyruğun başında ne olduğunu gösterir, kuyrukta değişiklik yapmaz.

Dairesel Kuyruk (Circular Queue)

- Normal dizide baştaki eleman çıkınca herkes bir öne kaydırılırsa her çıkışta n'lik bir iş yapılır. Bu verimli değildir.
- Bir öğrencinin önerdiği gibi yığındaki `top`'a benzer göstericiler tutulur. Bu dairesel kuyruk mantığıdır.
- Çıkan eleman silinmez. Yalnızca `front` bir sonraki elemana geçer. Boşalan göz yeni gelenler için kullanılabilir.
- Yeni eleman `rear`'ın gösterdiği yere yazılır ve `rear` ilerler. Dizinin sonuna gelince başa döner.

Örnek (ekrandan, 02:43:15), 5 gözlü dizi:

Durum	0	1	2	3	4	front	rear
Başlangıç	A	B	C	D	–	0	4
A bitti, E geldi	(A)	B	C	D	E	1	0
F geldi	F	B	C	D	E	1	1

- F gelince `front` ve `rear` aynı gözü gösterir. Hoca bunu "kuyruk dolu" olarak yorumladı. Bu durumda G gelirse "full" denir.
- Dairesel kuyruk sabit uzunluktadır. Dinamik ya da statik dizi kullanılabilir.

Bağlı Listeyle Kuyruk

- Kuyruk bağlı listeyle de gerçekleştirilebilir:
  - `head` kuyruğun başını, `last` sonunu gösterir.
  - Yeni gelen `last`'ın arkasına eklenir.
  - Hizmet `head`'in gösterdiği elemana verilir. Sonra `head` bir sonraki elemana geçer.
- Örnek: 3, 5, 7 sırayla gelir. `head` 3'ü, `last` 7'yi gösterir. 3'e hizmet bitince `head` 5'i gösterir.
- Dairesel kuyruk, ek gösterici tutmadan yalnızca `front` ve `rear` ile çalıştığı için güzel bir yapıdır.
- Kuyruğun ne kadar büyüyüp küçüleceği bilinmiyorsa ya da sınır konmak istenmiyorsa bağlı liste tercih edilebilir.

Ders sonrası notlar

- **Dairesel kuyrukta dolu/boş ayrımı:** derste `front` ve `rear` aynı gözü gösterince kuyruğun dolu olduğu söylendi. Ancak bu gösterimde `rear` bir sonraki boş gözü gösterdiği için kuyruk boşken de `front == rear` olur. Yalnızca bu eşitliğe bakarak dolu ve boş durumu ayırt edilemez. Yaygın iki çözüm vardır: kuyruktaki eleman sayısını ayrı bir sayaçta tutmak ya da bir gözü hep boş bırakıp `(rear + 1) % MAX == front` durumunu "dolu" saymak. Dairesel kuyruk fonksiyonlarını yazarken bunu hesaba katın.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.